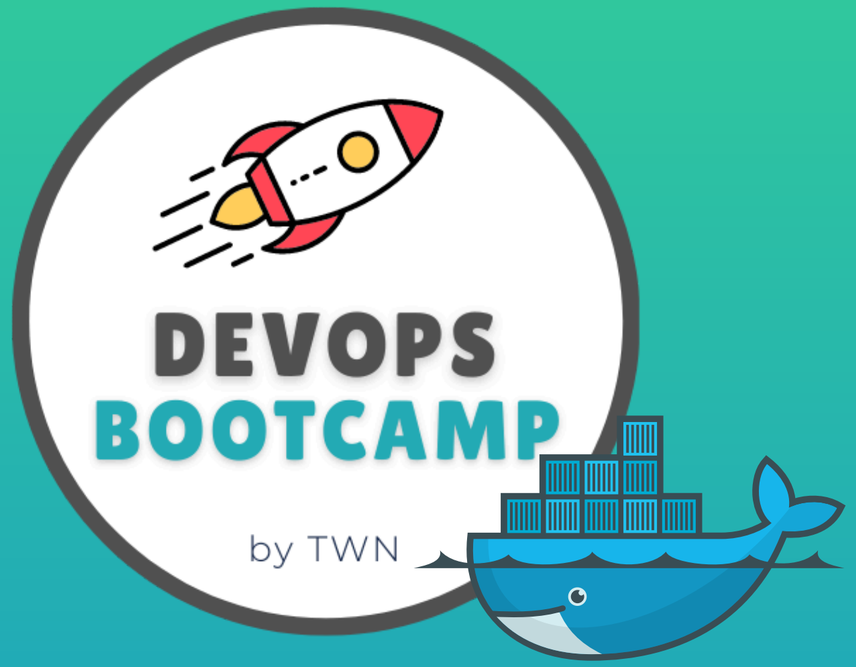


Containers with Docker

Key Takeaways

TECHWORLD WITH NANA

Introduction to Containers & Docker



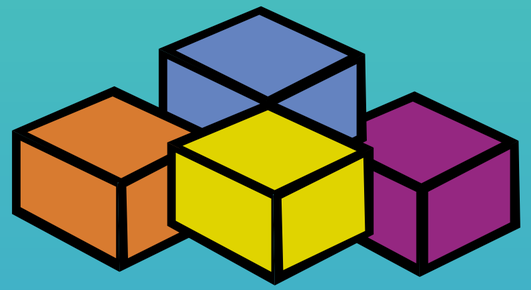
TECHWORLD WITH NANA

What is Docker?



- Docker is an open source **containerization platform**
- Enables developers to **package applications into containers**
- Containers existed already before Docker
- Docker made containers popular

What is a container?



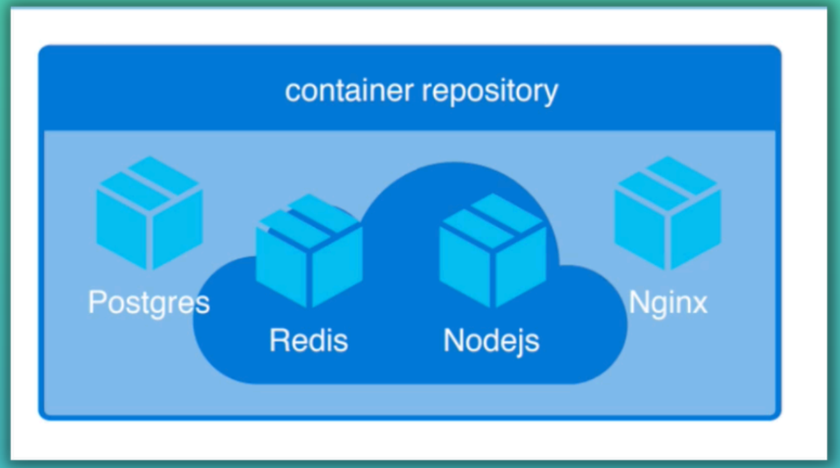
- A way to **package** an application with **all** the **necessary dependencies** and **configuration**
- **Portable standardized artifact** for development, shipment and deployment
- Makes development and deployment **more efficient**

TECHWORLD WITH NANA

Where container artifacts are hosted

- Hosted in **container repositories**
- There are private and public repositories
- Public repository for Docker:



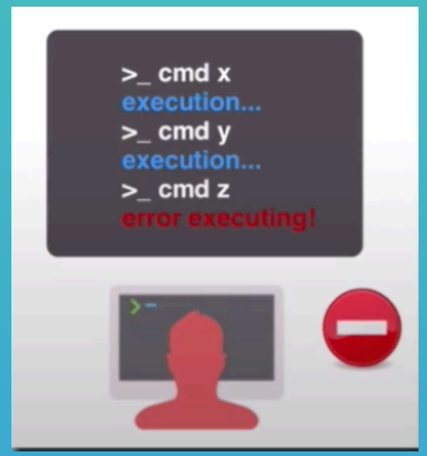


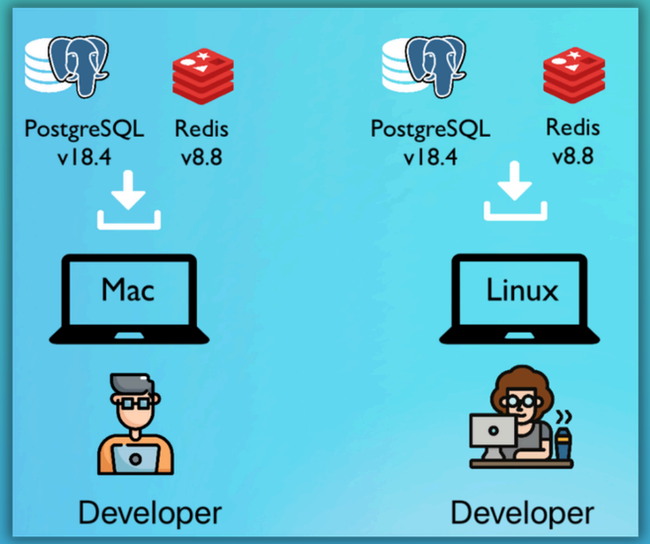
TECHWORLD WITH NANA

Application DEVELOPMENT before and after Docker

Before Containers

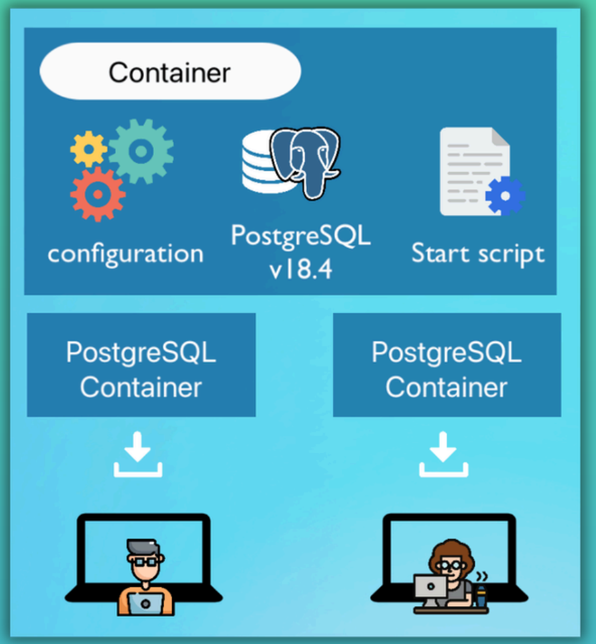
- **Installation process different** on each OS environment
- **Many steps** where something could go wrong



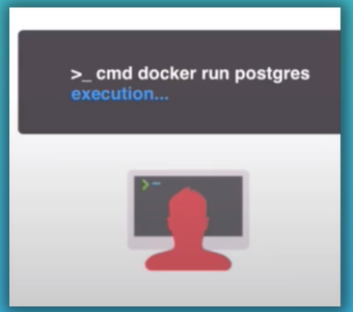


After Containers

- Own **isolated environment**
- Packaged with all needed configurations



- **1 command** to install the application
- Easily run same application with 2 different versions



TECHWORLD WITH NANA

Application DEPLOYMENT before and after Docker

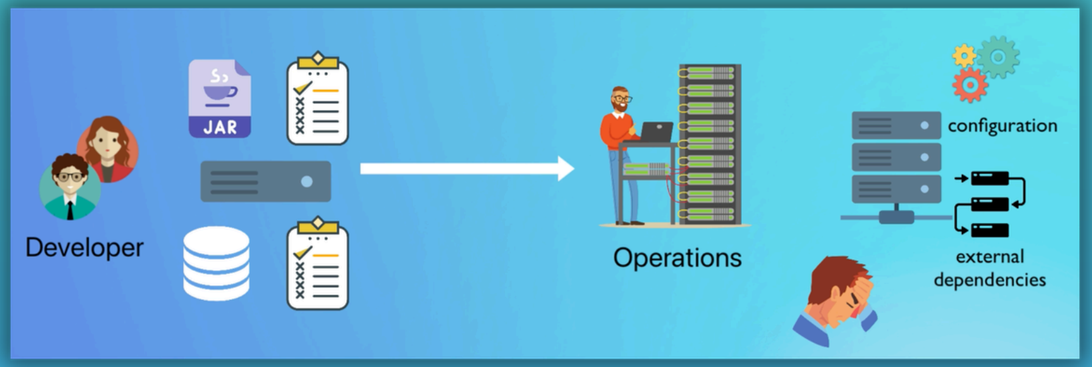
Before Containers

- Configuration on the server needed

❌ Dependency version conflicts

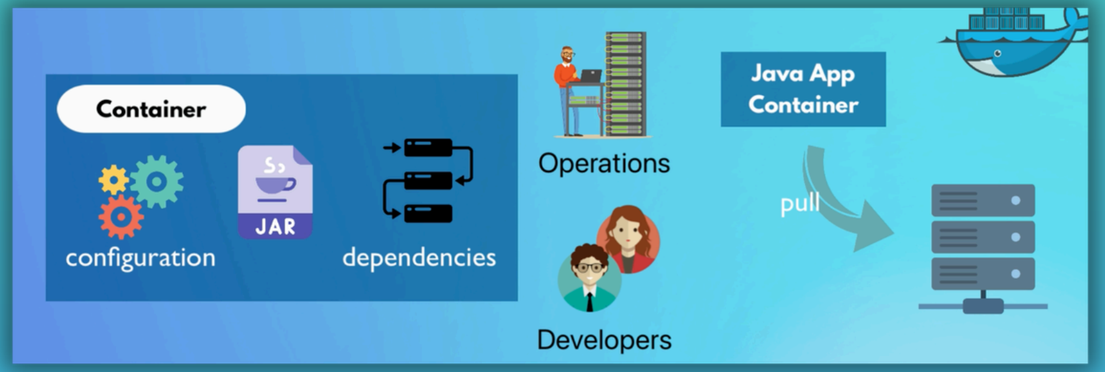
- Textual guide for deployment

❌ Misunderstandings



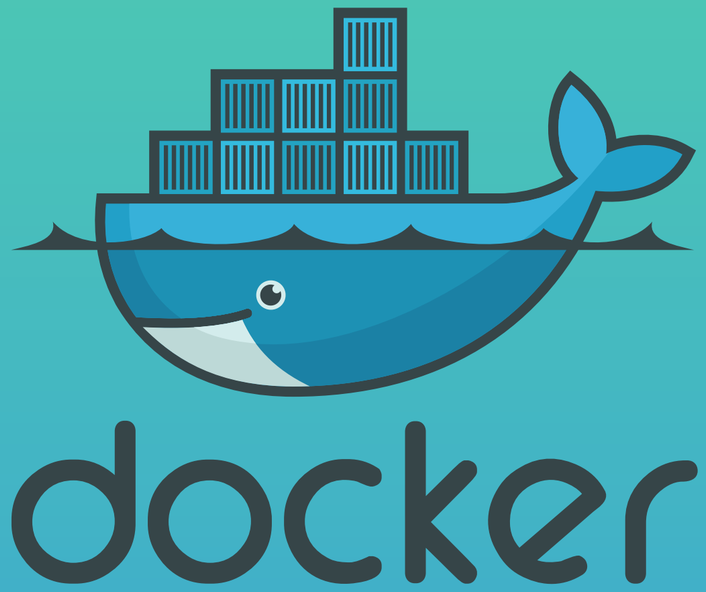
After Containers

- Devs & Ops work together to package the application in a container
- **No environment configuration needed** on server (except Container Runtime)



TECHWORLD WITH NANA

Docker = most popular container technology



Others:

containerd

cri-o

TECHWORLD WITH NANA

Docker Image vs Docker Container

Docker **Image**



- The actual package (file)
- **Artifact** that can be moved around
- Not in "running" state
- Consists of several layers
- Mostly Linux Base Image, application image on top





Images become containers at runtime

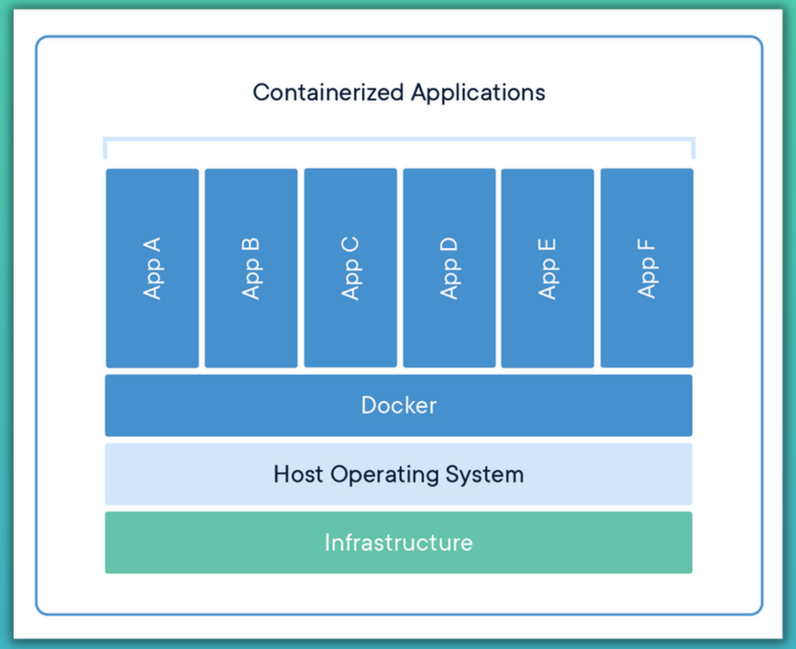
Docker **Container**



- Actually **starts the application**
- Is a **running environment** defined in the image
- Virtual file system
- Port binding: talk to application running inside of container

TECHWORLD WITH NANA

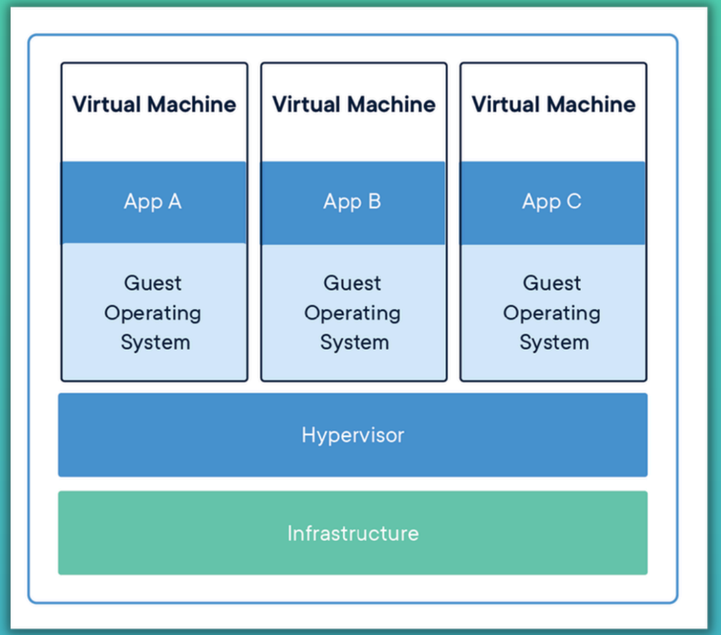
Containers vs Virtual Machines



Size: Docker Image much smaller

Speed: Docker containers start and run much faster

Compatibility: VM of any OS can run on any OS host



- Abstraction at the app layer
- Multiple containers share the OS kernel

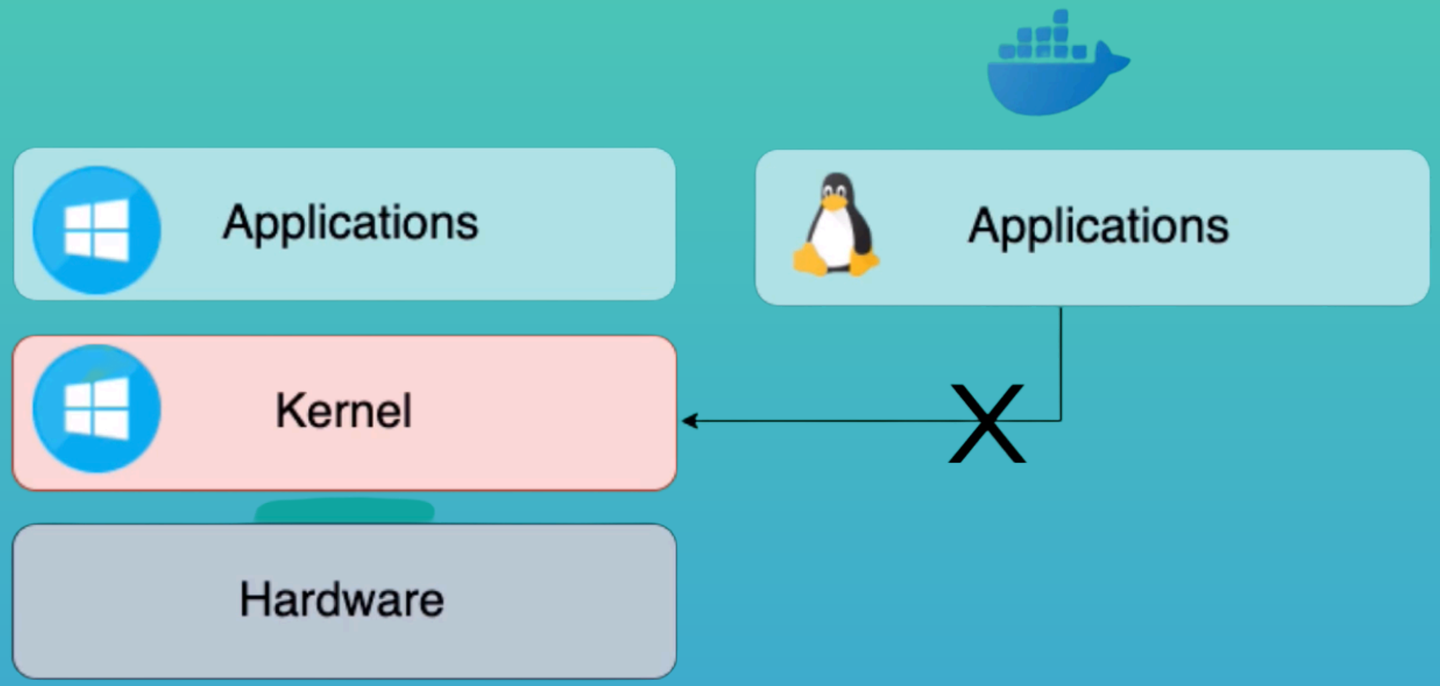
- Abstraction of physical hardware
- Each VM includes a full copy of an OS

Both are **virtualization** tools

TECHWORLD WITH NANA

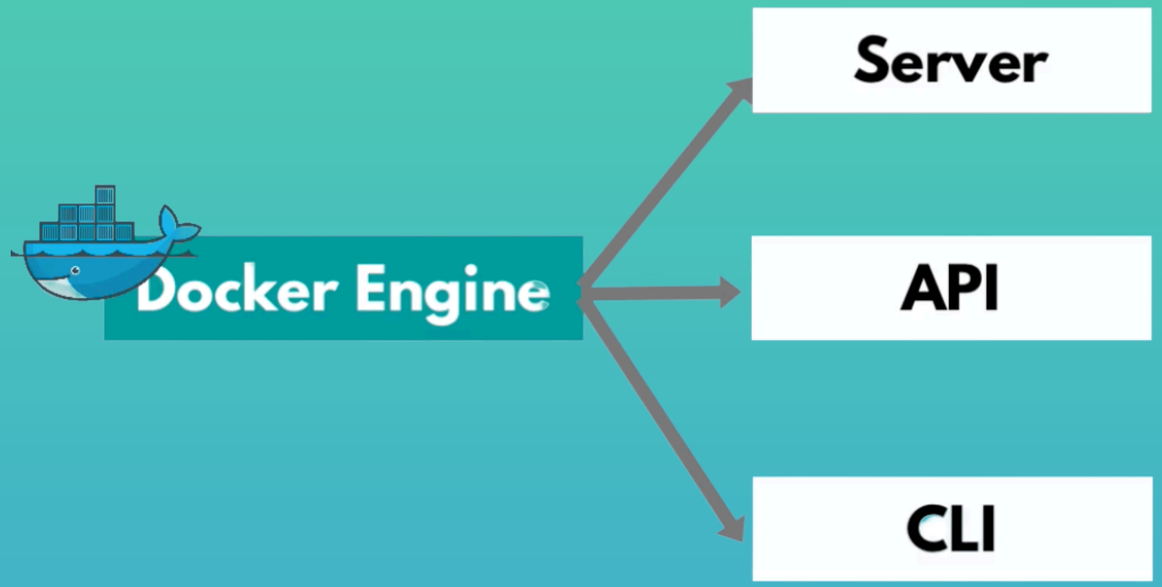
You can't run Linux container on a Windows host, but for that there is

Docker Desktop for Windows and Mac



TECHWORLD WITH NANA

Docker Architecture & its components - 1



- Pulling images
- Managing images & containers

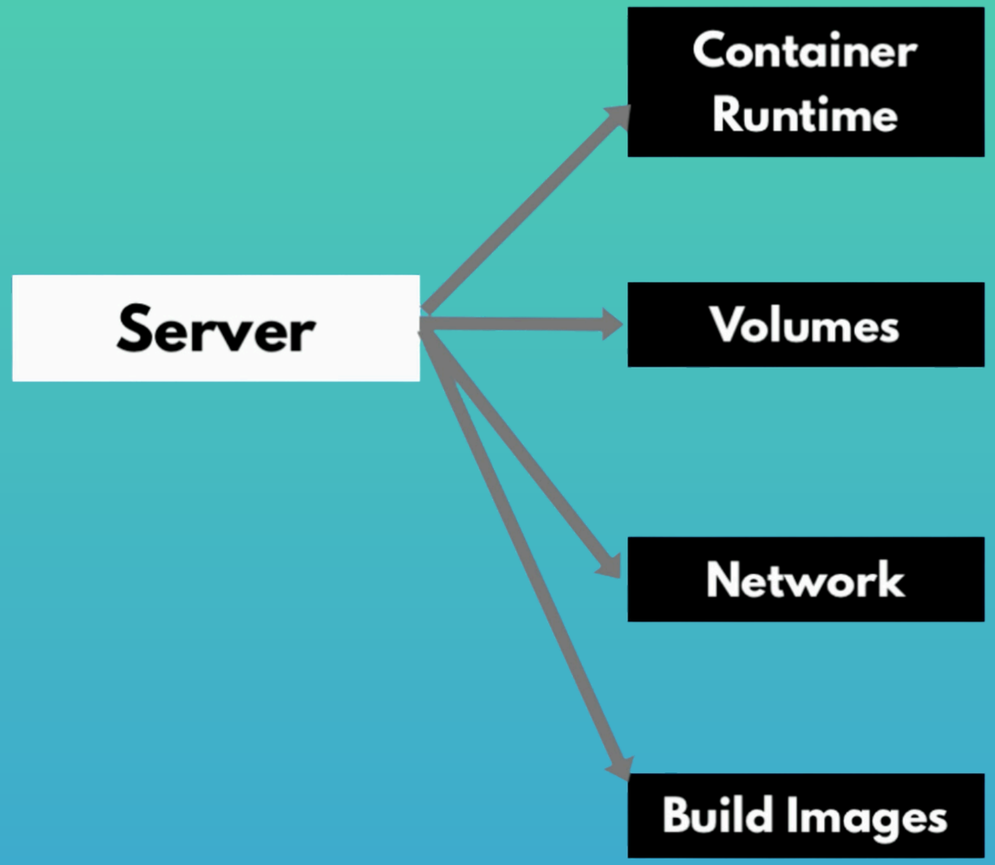
- Interacting with Docker Server

- Command Line Interface:

  Client to execute Docker commands

TECHWORLD WITH NANA

Docker Architecture & its components - 2



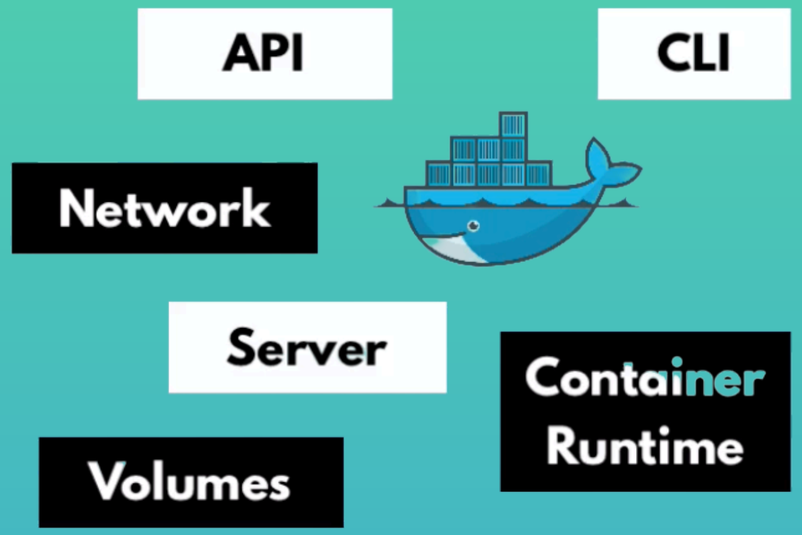
- Pulling images
- Managing container lifecycle

- Persisting data

- Configuring network for container communication

TECHWORLD WITH NANA

Docker Architecture & its components - 3



Docker has many functionalities in 1 application

Alternatives, if you need **only** a **container runtime**?

containerd

cri-o

Need to build an image?

buildah

TECHWORLD WITH NANA

Deep Dive into Docker



TECHWORLD WITH NANA

Docker Installation



Linux

native support

Mac + Windows

Docker Desktop



TECHWORLD WITH NANA

Main Docker Commands

```
docker run {image}
docker pull {image}
docker start {container}
docker stop {container}
docker images
docker ps
docker ps -a
```

1. **docker run**: creates a container from an image
2. **docker pull**: pull images from the docker repository
3. **docker start**: starts one or more stopped container(s)
4. **docker stop**: stops a running container
5. **docker images**: lists all the locally stored docker images
6. **docker ps:** lists the running containers
7. **docker ps -a:** show all the running and exited containers

Debug Commands

1. **docker logs**: fetch logs of a container
2. **docker exec -it**: creates a new bash session in the container

TECHWORLD WITH NANA

Ports in Docker - 1

- **Multiple containers** can run on your host machine

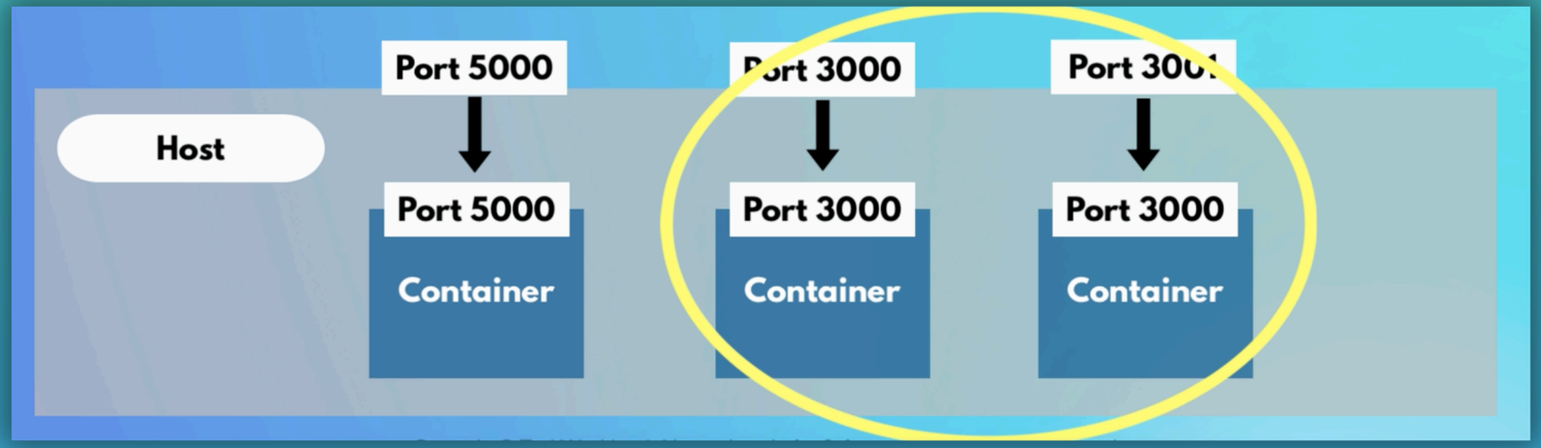
Problem:

- But your laptop has only certain ports available
- **Conflict when same port** on host machine, so we need to map to a free port on host machine:

Port Mapping:

```
docker run -p 8080:8080 {image}
```

- host port
- container port

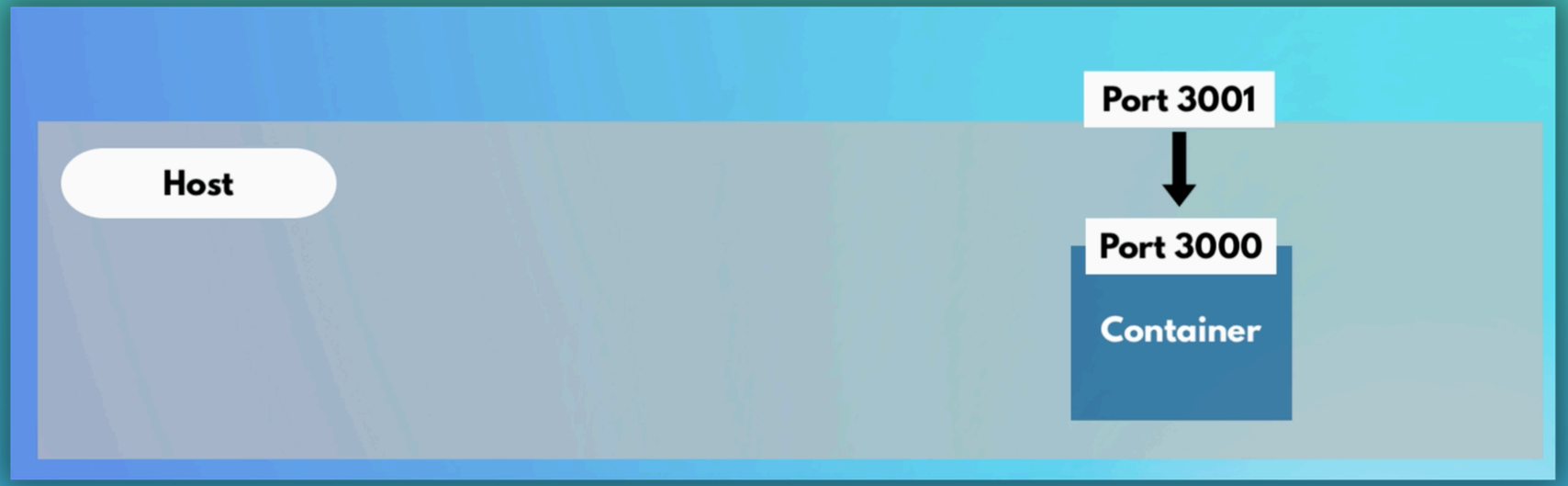


TECHWORLD WITH NANA

Ports in Docker - 2

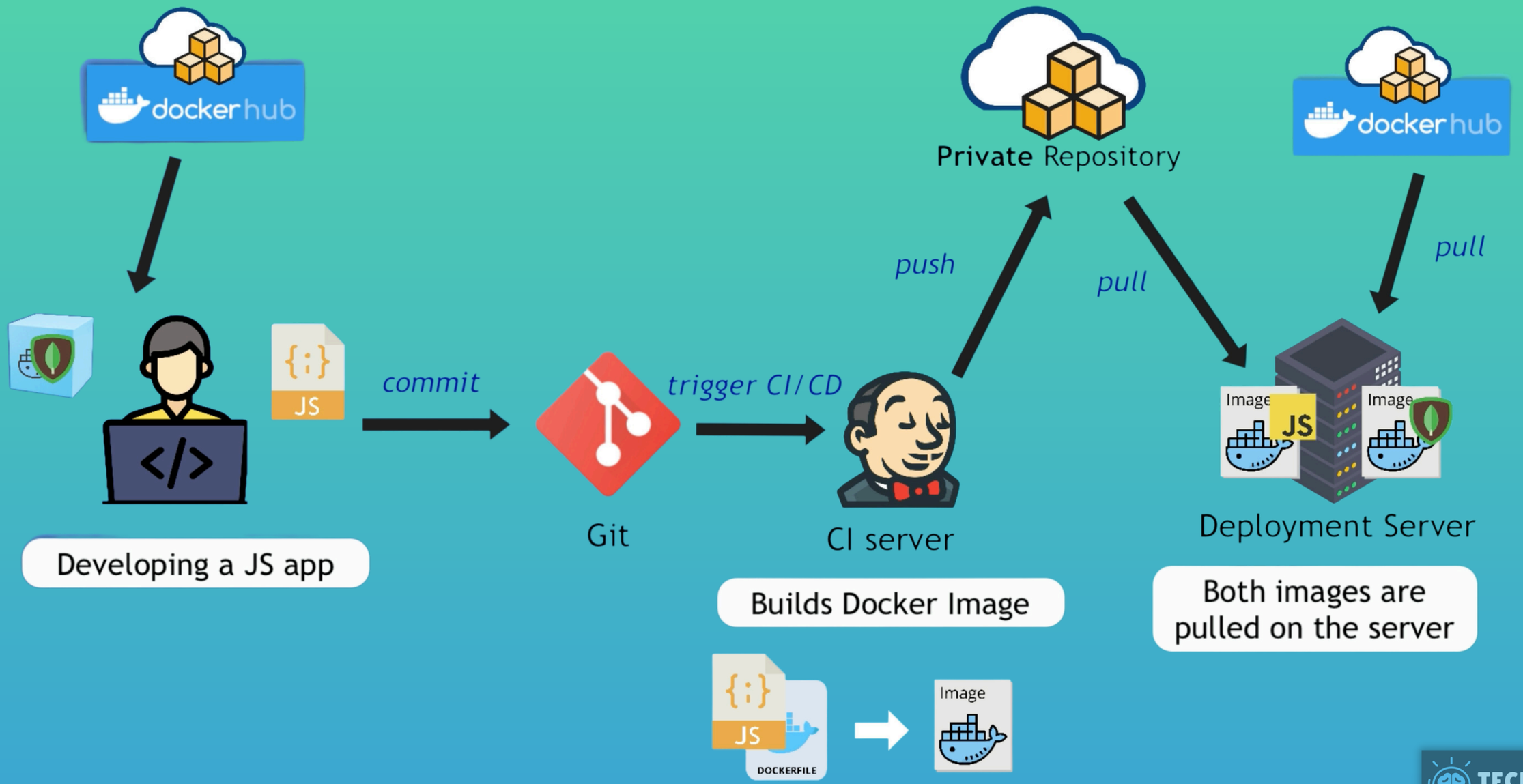
- **Container Port** = Port used in container
- **Host Port** = Port on the host machine

some-app://localhost:3001



TECHWORLD WITH NANA

Workflow with Docker



TECHWORLD WITH NANA

Docker Compose - 1

Docker Compose is a tool for defining and **running multiple docker containers**

- YAML file to configure your application's services:
- You can **maintain and update configuration more easily** than with *docker run* command

docker run command

```
docker run -d \
--name mongo-express \
-p 8081:8081 \

-e ME_CONFIG_MONGODB_
   URL=mongodb://admin:password@mongodb:27017 \
-e ME_CONFIG_BASICAUTH_
   USERNAME=user \
-e ME_CONFIG_BASICAUTH_
   PASSWORD=pass \

--net mongo-network \
techworldwithnana/mongo-express
```



mongo-docker-compose.yaml

```
services:
  mongodb:
    image: mongo
    ...
  mongo-express:

    image: techworldwithnana/mongo-express
    ports:
      - 8081:8081
    environment:
     - ME_CONFIG_MONGODB_U ....
     ...
```

- Docker Compose automatically creates a **common docker network** for docker containers in it (*--net* option in docker run)

TECHWORLD WITH NANA

Docker Compose - 2

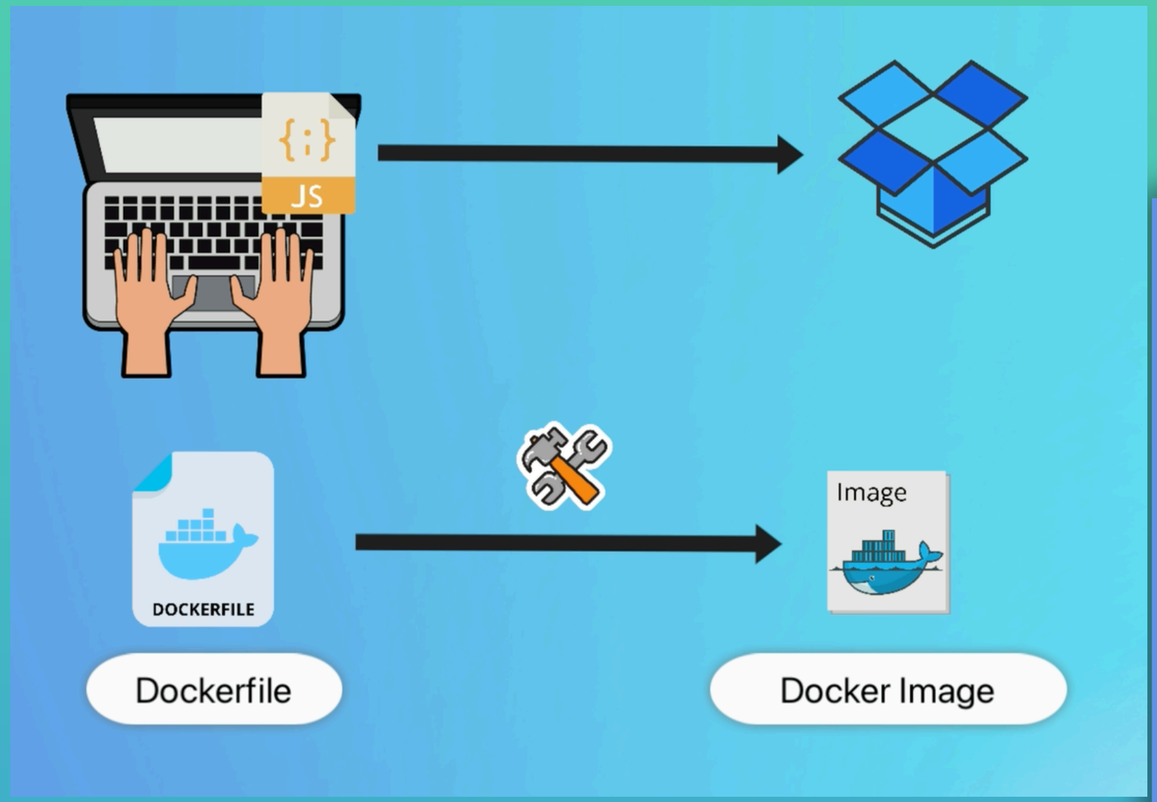
Example docker-compose.yaml file

```yaml
services:
  mongodb:
    image: mongo
    ports:
      - 27017:27017
    environment:
      - MONGO_INITDB_ROOT_USERNAME=admin
      - MONGO_INITDB_ROOT_PASSWORD=password
  mongo-express:
    image: techworldwithnana/mongo-express
    ports:
      - 8081:8081
    restart: always
    environment:
      - ME_CONFIG_MONGODB_URL=mongodb://admin:password@mongodb:27017
      - ME_CONFIG_BASICAUTH_USERNAME=user
      - ME_CONFIG_BASICAUTH_PASSWORD=pass
```

TECHWORLD WITH NANA

Dockerfile - 1

A simple text file that consists of **instructions to build Docker images**



- Some common Dockerfile commands:

Image Environment Blueprint	DOCKERFILE
install node	**FROM** node
set MONGO_DB_USERNAME=admin set MONGO_DB_PWD=password	**ENV** MONGO_DB_USERNAME=admin \ MONGO_DB_PWD=password
create /home/app folder	**RUN** mkdir -p /home/app
copy current folder files to /home/app	**COPY** . /home/app
start the app with: "node server.js"	**CMD** ["node", "server.js"]

CMD = entrypoint command

You can have multiple RUN commands

TECHWORLD

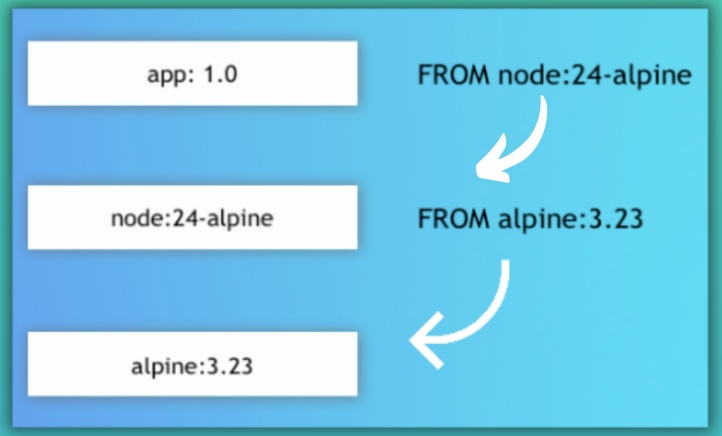
TECHWORLD WITH NANA

Dockerfile - 2

- Each instruction in a Dockerfile results in an Image Layer:



- It's common to start with an existing base image in your application's Dockerfile:



- Command to build an image from a Dockerfile and a context
- The build's context is the set of files at a specified location

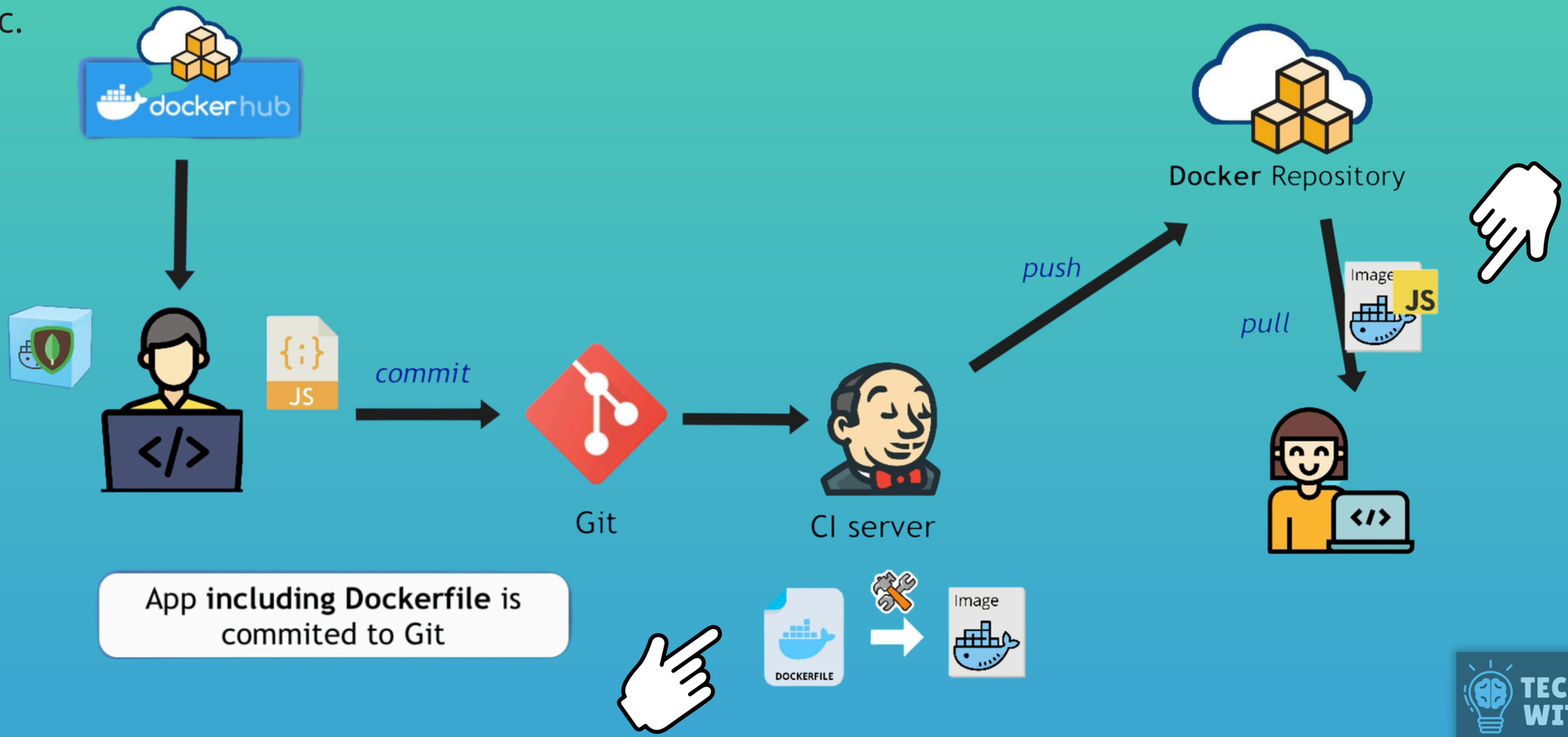
```
docker build -t my-app:1.0 .
```

Uses the current directory (.) as build context

TECHWORLD WITH NANA

Dockerfile - 3

- Dockerfile is **used in CI/CD** to build the Docker image artifact, which will be pushed to Docker repo,
- Docker image can then be pushed to multiple remote servers or pulled locally for development and testing etc.



TECHWORLD WITH NANA

Private Docker Repository - 1

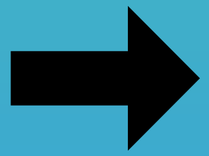
- When you work in a company, you will be working with a private docker registry
- **Public = DockerHub**, Example for **Private** = Amazon Elastic Container Registry "**AWS ECR**"

Difference to DockerHub

1. You need to **login,** so authenticate with the registry before fetching or pushing the image
2. **Tag** your image with the **registry address and name**,
3. Push the tagged image

```
docker login
- reponame
- username
- password

docker tag {repo-name}:{image-version}
docker push {tagged-image}
```



registryDomain/imageName:tag

TECHWORLD WITH NANA

Private Docker Repository - 2

In DockerHub

- **docker.io is default** repository

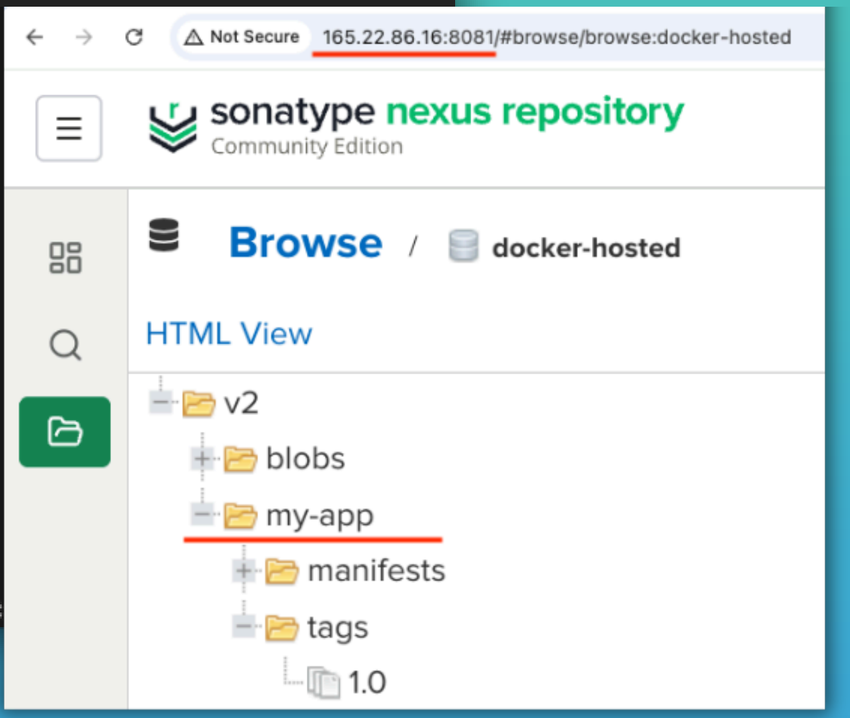
`docker pull mongo:8.3` = `docker pull docker.io/library/mongo:8.3`

In Nexus or any other private registry

- Need to include the registry domain

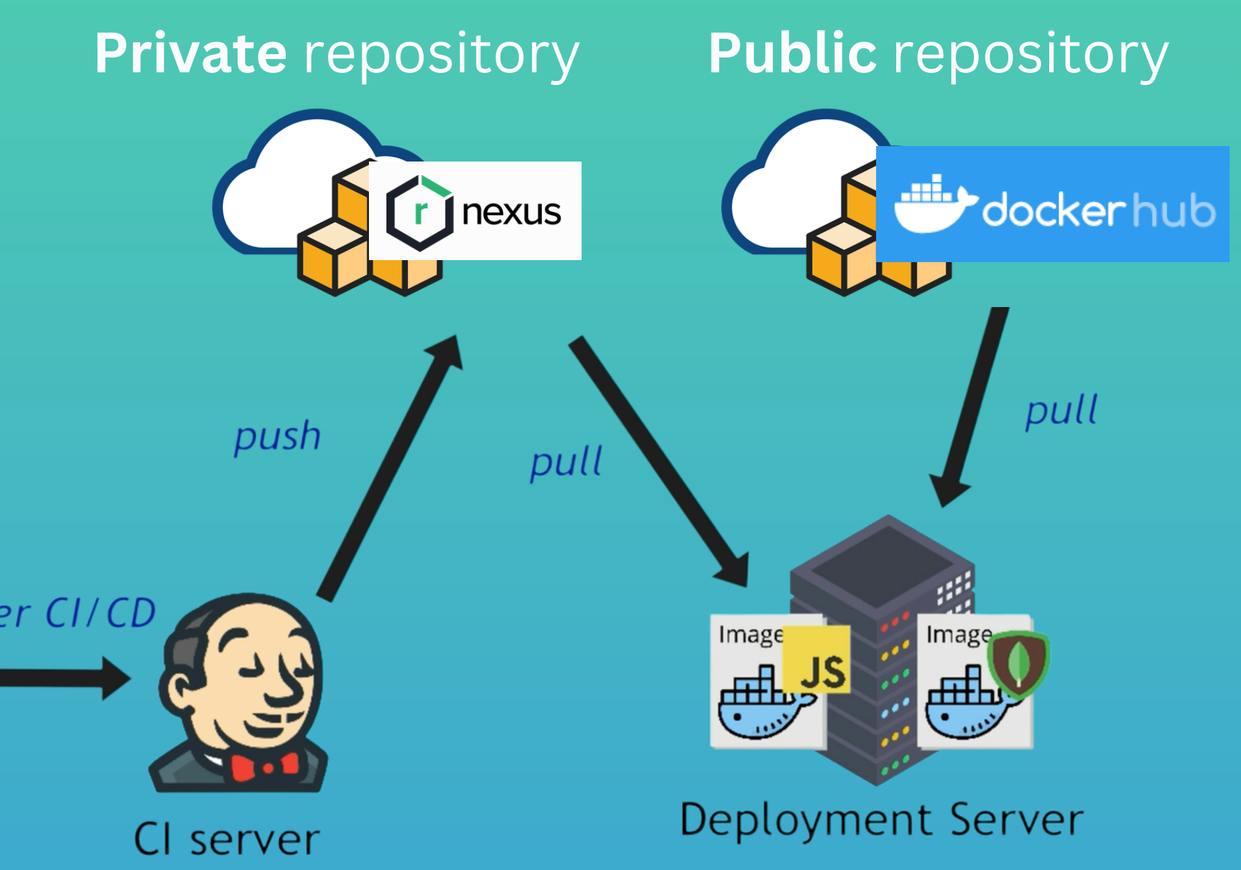
`docker pull 165.22.86.16:8083/my-app:1.0`

```
% docker tag my-app:1.0 165.22.86.16:8083/my-app:1.0
nana@macbook /Users/nana
% docker images | grep my-app
165.22.86.16:8083/my-app:1.0          d33d1fb18d89        283MB
my-app:1.0                            d33d1fb18d89        283MB
nana@macbook /Users/nana
% docker push 165.22.86.16:8083/my-app:1.0
The push refers to repository [165.22.86.16:8083/my-app]
51d3fd210d57: Pushed
8e7170666875: Pushed
4f4c69389a30: Pushed
4f4fb700ef54: Pushed
1220594deafa: Pushed
0d79637ec254: Pushed
5de55e5ef9c0: Pushed
d9f8fbd0bc86: Pushed
c480ce1f559d: Pushed
1.0: digest: sha256:d33d1fb18d89746605758d192cdb12374a4acdd958779f66c
```



TECHWORLD WITH NANA

Deploy Docker Containers on a Remote Server



On deployment server, you can pull from both:

Private Images (Your JS application, company internal libraries)

↓

from **private docker repository** (like AWS ECR, Azure Container Registry etc.)

Public Images (Mysql, MongoDB)

↓

from **public docker repositories** (like DockerHub)

TECHWORLD WITH NANA

Docker Volumes - 1

Volumes are the way to **persist data** generated by and used by Docker containers

Why are volumes needed for persistence?

- Data is stored on the virtual file system of the container. So when container is removed, the data is deleted as well.

Important for

- Databases
- Other stateful applications

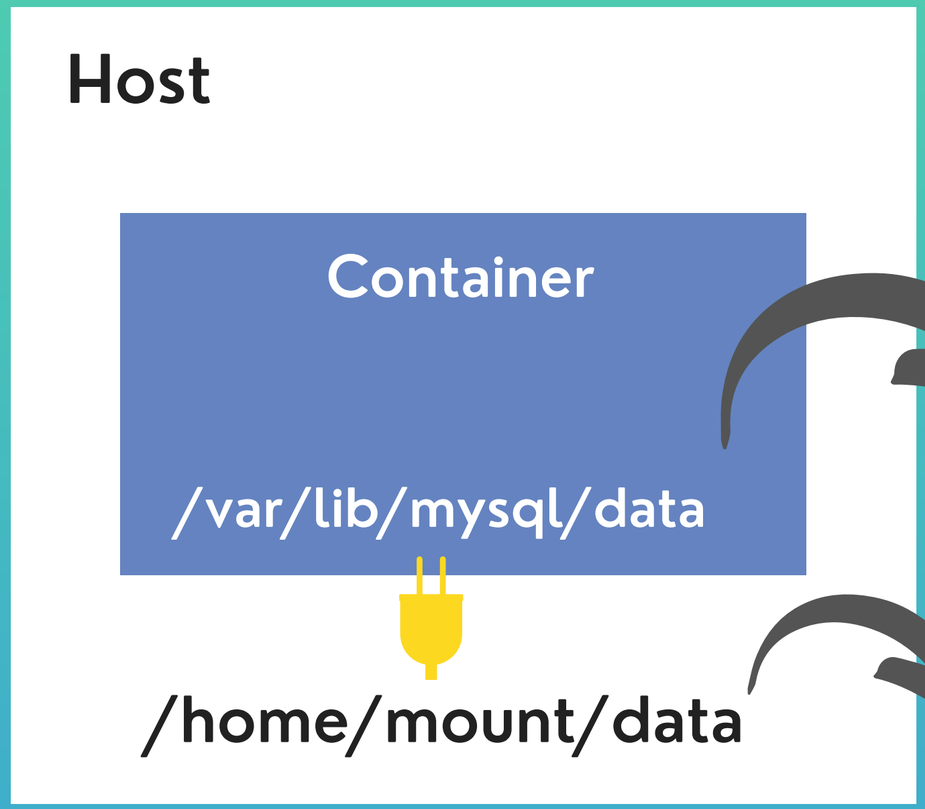


By default:
Data is gone when removing the container

TECHWORLD WITH NANA

Docker Volumes - 2

How Docker Volumes work



- Folder in physical host file system is **mounted** into the virtual file system of Docker
- Data gets **automatically replicated**

Virtual File System inside container

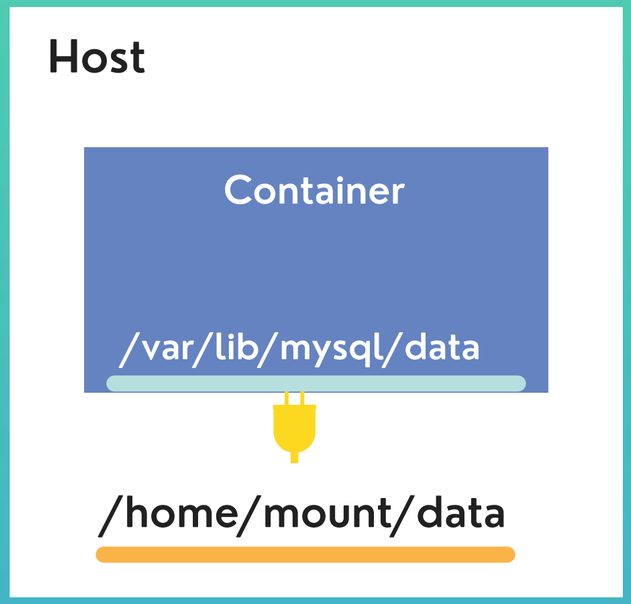
Host File System (physical)

TECHWORLD WITH NANA

Docker Volumes - 3

3 Volume Types

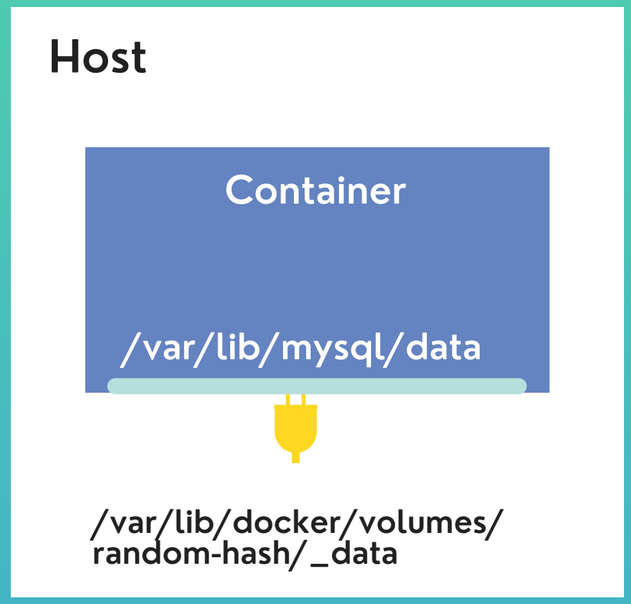
Host Volumes



- You decide **where on the host file system** the reference is made

```
docker run
-v /home/mount/data:/var/lib/mysql/data
```

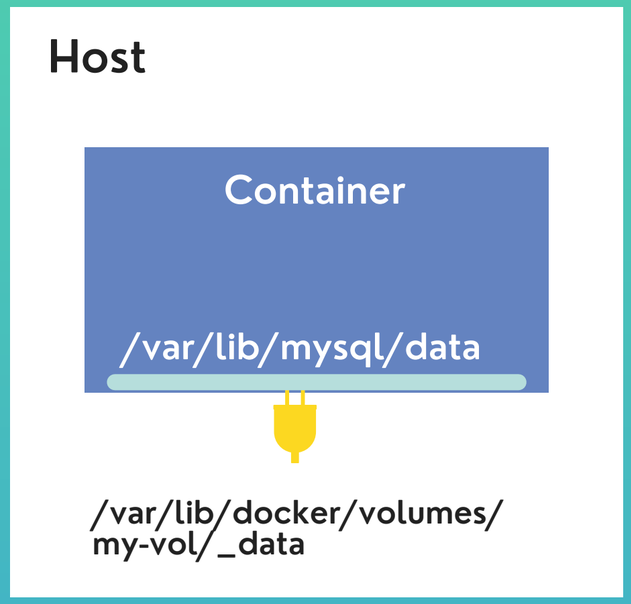
Anonymous Volumes



- For **each container a folder is generated** that gets mounted

```
docker run
-v /var/lib/mysql/data
```

Named Volumes



- You can **reference** the volume **by name**
- Should be used in production

```
docker run
-v name:/var/lib/mysql/data
```

Docker Volumes - 4

Docker Volumes in docker-compose file:

mongo-docker-compose.yaml



Docker Best Practices - 1

- **Security**: Only use official/trusted Docker images as base image to avoid malware
- **Security**: Use specific image versions
- **Size & Security**: Use minimal base images (e.g. prefer alpine-based images over full-fledged system OS images)
- **Size**: Optimize caching image layers
- **Size**: Use .dockerignore to exclude files and folders
- **Cleaner Dockerfile**: Make use of "Multi-Stage Builds"





Improve Security



Optimize Image Size



Write cleaner and more maintainable Dockerfiles

Docker Best Practices - 2



- **Security**: Use least privileged user (create a dedicated user and group with minimal permissions to run the application)
- **Security**: Scan your images for vulnerabilities
- **Security**: Don’t leak sensitive information to Docker images



Improve Security



Optimize Image Size



Write cleaner and more maintainable Dockerfiles